

Particionamento de dados e balanceamento de carga

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

“Escreva um código que simule o movimento de um fluido ao longo do tempo usando a equação de Navier-Stokes, considerando apenas os efeitos da viscosidade. Desconsidere a pressão e quaisquer forças externas. Utilize diferenças finitas para discretizar o espaço e simule a evolução da velocidade do fluido no tempo. Inicialize o fluido parado ou com velocidade constante e verifique se o campo permanece estável. Em seguida, crie uma pequena perturbação e observe se ela se difunde suavemente. Após validar o código, paralelize-o com OpenMP e explore o impacto das cláusulas `schedule` e `collapse` no desempenho da execução paralela.”

2 O cálculo comum às sete versões

A simulação suaviza uma perturbação em uma malha. A operação principal está nesta linha, em que `c` identifica uma célula e `tamanho_grid` é a largura da malha:

```
46         size_t c = (size_t)i*tamanho_grid+j;  
47         v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c  
           +tamanho_grid]-4*u[c]);
```

Listing 1: Leituras do campo atual e escrita no próximo campo.

A expressão lê cinco posições de `u` e escreve uma posição de `v`. As posições `c-1` e `c+1` são os vizinhos na mesma linha; `c-tamanho_grid` e `c+tamanho_grid` são os vizinhos nas linhas anterior e seguinte. Esse padrão é chamado de stencil de cinco pontos. A física determina a expressão, mas a possibilidade de paralelizar vem de onde ela lê e escreve.

3 Comparação direta entre as versões

As sete versões atualizam a mesma malha, com a mesma expressão e os mesmos passos. A `v0` é sequencial. As outras mantêm a mesma região paralela e as mesmas barreiras; as mudanças estão na política de distribuição e no uso de `collapse(2)`. Os trechos abaixo destacam apenas essas diferenças. O apêndice contém os arquivos completos.

Versão	Política	Unidade distribuída
v0	Sequencial	Sem distribuição
v1	static	Linha
v2	static, collapse(2)	Célula
v3	dynamic, 16	Bloco de 16 linhas
v4	dynamic, 16, collapse(2)	Bloco de 16 células
v5	guided, 16, collapse(2)	Bloco decrescente de células
v6	guided, 16	Bloco decrescente de linhas

Table 1: Versões completas e independentes. Em guided, 16 é o tamanho mínimo, salvo o último bloco.

3.1 v0 e v1: execução sequencial e divisão por linhas

```

1  /* v0: uma thread percorre todas as linhas. */
2  for (int i = 1; i < tamanho_grid-1; ++i) { ... }
3
4  /* v1: a equipe divide as linhas. */
5  #pragma omp for schedule(static)
6  for (int i = 1; i < tamanho_grid-1; ++i) { ... }

```

Listing 2: Mudança de v0 para v1, em forma resumida.

Na v0, uma thread faz todas as atualizações. Na v1, a região `parallel` mantém uma equipe, e o `omp for` entrega um bloco de linhas a cada thread. Quem recebe uma linha executa todas as suas colunas. O total de células calculadas é o mesmo; muda quem as calcula.

A v1 pode ganhar porque diferentes núcleos executam blocos ao mesmo tempo. A divisão estática combina com este código: não há células que executem um algoritmo mais caro que as demais. Cada linha tem aproximadamente o mesmo custo, então a atribuição inicial já tende a equilibrar a equipe.

A v1 também paga custos que a v0 não tem: entrada na região paralela, distribuição das linhas e sincronização entre passos. Com pouco trabalho por thread, esses custos podem consumir o ganho. Usar uma thread OpenMP mantém essa estrutura sem repartir o cálculo entre núcleos; por isso, não garante o mesmo tempo da versão sequencial.

3.2 v1 e v2: static por linhas ou por células

```

1  /* v1 */
2  #pragma omp for schedule(static)
3  /* v2 */
4  #pragma omp for collapse(2) schedule(static)

```

Listing 3: A única mudança no escalonamento entre v1 e v2.

A v1 divide as iterações de *i*. A v2 associa os laços de *i* e *j* e distribui pares de índices. Imagine uma malha didática com duas linhas internas e várias colunas, executada por quatro threads.

Sem collapse, só existem duas iterações externas para distribuir, então parte da equipe fica sem trabalho. Com collapse, há várias células para dividir entre as quatro threads.

Na malha desta tarefa, já existem 510 linhas internas, mais que as threads usadas. A v2 não precisa de collapse para ocupar a equipe. Ela muda a organização do laço sem obter aquela vantagem do exemplo pequeno.

Static continua dividindo o espaço de iterações em blocos. Na v2, uma célula é uma iteração lógica, mas não exige uma solicitação ao escalonador para cada atualização. Portanto, o custo de muitos pedidos de trabalho, discutido para a v4, não explica automaticamente a diferença entre v1 e v2.

Na v1, o compilador encontra um laço interno simples que percorre colunas contíguas. Collapse pode alterar a geração dos índices e a vetorização desse percurso. Para atribuir a diferença de tempo a uma dessas causas, seria preciso examinar o código gerado ou os diagnósticos de vetorização. O gráfico mostra que v2 ficou acima de v1, mas não separa esses efeitos.

3.3 v1 e v3: divisão fixa ou entrega de novos blocos

```
1 /* v1 */
2 #pragma omp for schedule(static)
3 /* v3 */
4 #pragma omp for schedule(dynamic, 16)
```

Listing 4: Comparação mantendo a linha como unidade.

Ambas distribuem linhas. Na v1, cada thread recebe sua parte e a executa. Na v3, uma thread termina um bloco de até 16 linhas e solicita outro. Para entregar blocos distintos às threads, o ambiente de execução precisa coordenar quais ainda estão disponíveis.

A v3 pode ganhar se algumas threads terminarem antes e assumirem trabalho que, em uma divisão fixa, ficaria preso a uma thread mais lenta. Mas as células deste stencil têm custo uniforme. Nesse caso, fazer novos pedidos pode acrescentar mais custo do que o tempo de espera que evita.

Esta comparação muda a política e o tamanho dos blocos: v1 não fixa chunk, enquanto v3 fixa 16 linhas. Ela não permite atribuir toda diferença apenas ao nome static ou dynamic. A política e a granularidade trabalham juntas.

3.4 v3 e v4: o mesmo chunk entrega quantidades diferentes de cálculo

```
1 /* v3 */
2 #pragma omp for schedule(dynamic, 16)
3 /* v4 */
4 #pragma omp for collapse(2) schedule(dynamic, 16)
```

Listing 5: Collapse muda o significado do chunk.

Com chunk 16, a v3 entrega até 16 linhas por solicitação. Cada linha contém 510 células internas. A v4 entrega até 16 células. O cálculo é o mesmo, mas a v4 retorna ao escalonador depois de muito menos trabalho útil.

Para visualizar a diferença sem recorrer aos tempos medidos, imagine quatro linhas de seis células e chunk dois. Sem collapse, um bloco reúne duas linhas, ou 12 atualizações; bastam dois blocos para percorrer a malha. Com collapse, cada bloco reúne duas células; são necessários 12 blocos. O número de threads não mudou. O número de entregas de trabalho aumentou.

Cada entrega exige coordenação. Se o bloco contém poucas operações, a equipe passa uma parcela maior da execução obtendo trabalho em vez de calculá-lo. Isso torna o custo de escalonamento uma explicação compatível com a curva alta da v4. Blocos de linhas, como os da v3, diluem esse custo em mais atualizações.

Para investigar o efeito de collapse sem essa diferença de volume, seria necessário comparar chunks que representassem quantidades equivalentes de células. A comparação atual mostra o efeito da combinação `dynamic, 16` com collapse, não uma desvantagem universal de collapse.

3.5 v4 e v5: blocos pequenos fixos ou blocos decrescentes

```
1 /* v4 */
2 #pragma omp for collapse(2) schedule(dynamic, 16)
3 /* v5 */
4 #pragma omp for collapse(2) schedule(guided, 16)
```

Listing 6: Comparação mantendo collapse nas duas versões.

As duas distribuem células, mas o valor 16 tem papéis diferentes. Em `dynamic`, ele fixa o tamanho dos blocos, salvo o último. Em `guided`, ele define o mínimo, salvo o último. A v5 pode entregar blocos grandes no começo, quando há muito trabalho disponível, e blocos menores perto do fim.

Essa diferença reduz a necessidade de solicitar blocos pequenos desde o início. Por isso, `guided` pode amortizar parte do custo que pesa na v4, sem abandonar a redistribuição durante o laço. A curva da v5 abaixo da v4 é coerente com esse mecanismo.

`Guided` não elimina toda coordenação e não garante equilíbrio perfeito. Uma thread que recebe um bloco caro continua responsável por ele. A política ajusta as próximas entregas, não transfere automaticamente um bloco que já está em execução.

3.6 v5 e v6: guided por células ou por linhas

```
1 /* v5 */
2 #pragma omp for collapse(2) schedule(guided, 16)
3 /* v6 */
4 #pragma omp for schedule(guided, 16)
```

Listing 7: Comparação mantendo guided nas duas versões.

A v5 usa células como iterações; a v6 usa linhas. Nas duas, os blocos diminuem. Ao chegar perto do tamanho mínimo, porém, um bloco mínimo completo da v6 reúne 16 linhas, enquanto o da v5 reúne 16 células. O último bloco pode ser menor nas duas versões.

A v6 conserva um laço interno de colunas contíguas e mais cálculo por bloco mínimo. A v5 permite entregas menores no final, o que pode ajudar a distribuir o trabalho restante, mas exige

mais coordenação se muitos blocos pequenos forem necessários. O ganho depende de qual custo pesa mais: deixar trabalho concentrado em uma thread ou distribuir muitos blocos.

Para comparar guided com static sem mudar para células, o par é v6 e v1. Para comparar guided com dynamic mantendo linhas e o parâmetro de chunk, o par é v6 e v3. Isso evita atribuir a uma política uma diferença que também envolve collapse.

4 O que permanece igual nas versões paralelas

4.1 Leituras compartilhadas e escritas independentes

Imagine uma thread calculando $v[c]$ e outra calculando $v[c+1]$. As duas consultam valores de u em comum, mas nenhuma modifica u durante esse passo. As escritas vão para posições distintas de v . Não há conflito de leitura com escrita nem de escrita com escrita entre essas duas iterações.

Esse é o argumento para usar `omp for`: as atualizações espaciais do mesmo passo são independentes. Compartilhar memória não exige um lock em todo acesso. O que exige atenção é o acesso concorrente à mesma posição quando pelo menos uma operação escreve nela.

4.2 Dois vetores evitam misturar passos

Se a primeira thread sobrescrever $u[c]$ antes de a segunda ler esse vizinho, a segunda usará um valor novo junto de outros valores antigos. Se a ordem das threads mudar, ela poderá ler o valor antigo. O resultado passa a depender da ordem de execução e deixa de representar a atualização que usa apenas o campo anterior.

Os dois vetores separam os instantes: u é a entrada deste passo e v é a saída. Depois de concluir todas as células, o programa troca os ponteiros. Essa troca muda o papel dos vetores sem copiar a malha inteira.

4.3 A dependência entre passos impede paralelizar o tempo

O passo seguinte lê o campo produzido pelo passo atual. Há uma dependência de leitura após escrita, ou RAW. Distribuir passos diferentes entre threads permitiria ler um campo que ainda não está pronto. Por isso, `collapse(2)` se aplica aos laços espaciais de i e j , nunca ao laço temporal desta implementação.

4.4 A execução de um passo

```
38 #pragma omp parallel default(none) shared(u,v,tamanho_grid,
    num_passos_tempo,r,threads)
39 {
40     #pragma omp single
41     threads = omp_get_num_threads();
42     for (int t = 0; t < num_passos_tempo; ++t) {
43         #pragma omp for schedule(static)
44         for (int i = 1; i < tamanho_grid-1; ++i) {
45             for (int j = 1; j < tamanho_grid-1; ++j) {
```

```

46         size_t c = (size_t)i*tamanho_grid+j;
47         v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
           +tamanho_grid]-4*u[c]);
48     }
49 } /* Barreira: todas as escritas terminam antes da troca. */
50 #pragma omp single
51 {
52     double *tmp = u; u = v; v = tmp;
53 } /* Barreira: todas as threads veem a troca antes do próximo
    passo. */
54 }
55 }

```

Listing 8: Região paralela completa da versão v1.

4.5 parallel forma a equipe; omp for divide as iterações

`parallel` forma a equipe. Todas as threads executam o bloco delimitado pelas chaves. `omp for` divide entre essa equipe as iterações do laço que vem logo depois. Sem `omp for`, cada thread percorreria todas as linhas, repetiria o cálculo e escreveria nas mesmas posições das demais.

No código, cada thread executa seu próprio laço `t`, mas recebe apenas parte das iterações de `i`. Todas participam do mesmo passo temporal. As barreiras impedem que uma avance para o próximo passo enquanto outra ainda termina o atual.

A equipe permanece ativa ao longo de todos os passos porque `parallel` está fora do laço `t`. Colocar uma região paralela dentro desse laço repetiria o custo de entrar e sair da região a cada passo, mesmo que o ambiente de execução reaproveitasse as threads.

4.6 A ordem das operações em duas threads

Considere duas threads, A e B, responsáveis por blocos distintos de linhas:

1. A e B leem `u` e preenchem suas posições de `v`.
2. A termina primeiro e espera na barreira do `omp for`. B ainda está escrevendo sua parte.
3. Quando B termina, ambas podem chegar ao `single`. Apenas uma executa a troca dos ponteiros.
4. A barreira ao final de `single` só libera a equipe depois da troca. O próximo passo lê o vetor que acabou de ser preenchido.

A thread que executa `single` não precisa ser a thread de número zero, nem precisa ser a mesma em todos os passos.

4.7 Cada barreira protege uma etapa diferente

Elas protegem momentos diferentes. A primeira separa o preenchimento da saída da troca de ponteiros. A segunda separa a troca do início das próximas leituras.

Se o `omp for` tiver `nowait`, A poderá trocar os ponteiros enquanto B ainda usa `u` e `v` no cálculo. Além de o campo estar incompleto, B poderá observar os ponteiros alterados durante seu trabalho.

Se apenas o `single` tiver `nowait`, uma thread que não executa a troca poderá chegar ao próximo passo antes de ela terminar. A primeira barreira, sozinha, não garante que a troca já aconteceu. Nesta organização, retirar qualquer uma delas quebra a ordenação necessária.

4.8 `single` executa uma vez; `critical` executaria várias vezes

A troca deve acontecer uma única vez por passo. `single` escolhe uma thread para executá-la. `critical` permitiria que todas a executassem, uma após a outra; ele garante exclusão mútua, não execução única. A equipe faria várias trocas em vez de uma.

`atomic` protege uma operação de memória compatível com a diretiva. Ele não transforma as três atribuições da troca em uma operação única para a equipe, nem espera todas as células ficarem prontas. `reduction` combina resultados parciais, como somas, e não representa a troca de dois ponteiros.

Dentro do stencil, cada posição de saída tem um único escritor. Colocar `critical` em cada atualização faria as threads disputarem uma região exclusiva sem resolver nenhum conflito necessário. A sincronização deve ficar na passagem de um passo para o outro.

4.9 Escopo dos ponteiros, dos vetores e dos índices

`u` e `v` são ponteiros compartilhados. Todos devem enxergar a mesma troca de papéis dos vetores. O escopo do ponteiro e o da memória apontada são questões distintas: tornar um ponteiro privado não cria uma cópia da malha e, com `private`, nem preserva automaticamente seu valor inicial.

O contador `t` é local a cada thread porque é declarado dentro da região paralela. O contador `i` do laço distribuído é privado; `j`, `c` e `tmp` também são locais. Assim, uma thread não altera o índice de outra. Os parâmetros usados no cálculo são compartilhados e apenas lidos.

`default(none)` obriga a explicitar o escopo das variáveis que não têm escopo predeterminado. Ele ajuda a revisar o código, mas não detecta nem elimina sozinho uma condição de corrida.

5 Memória e sincronização na leitura das curvas

5.1 Localidade: o que a divisão por linhas preserva

No acesso $c = i * N + j$, aumentar `j` em uma unidade avança um `double` na memória. Uma linha de cache de 64 bytes comporta oito valores desse tipo no ambiente usado. Trazer uma linha de cache pode atender vários acessos seguintes; o mesmo valor também participa de atualizações vizinhas do stencil.

Nas versões sem collapse, uma thread percorre as colunas da linha recebida antes de passar à próxima. Isso favorece acessos contíguos e reutilização local. Collapse não muda a disposição dos vetores na memória. Blocos contíguos de células também podem aproveitar cache; não basta ver collapse no pragma para concluir que a localidade foi perdida.

O problema aparece quando a distribuição fragmenta o trabalho entre threads e reduz a reutilização em cada núcleo. A política e o tamanho do bloco determinam essa fragmentação. O gráfico de tempo não mede diretamente os acertos e as faltas de cache.

5.2 Falso compartilhamento não é condição de corrida

Imagine duas posições de v na mesma linha de cache, escritas por threads em núcleos diferentes. Cada posição tem um único escritor, então o resultado pode estar correto. Ainda assim, a coerência de cache opera sobre a linha inteira, e as escritas podem exigir transferências de propriedade e invalidações.

Esse é o falso compartilhamento: disputa pelo bloco de cache sem disputa pelo mesmo elemento. Um lock não é a solução para esse custo. A investigação deve olhar a distribuição das escritas e os limites dos blocos. Nesta tarefa, não houve uma medição específica que permita apontá-lo como causa da diferença entre versões.

5.3 Por que as curvas param de melhorar?

No começo, mais threads podem reduzir o cálculo atribuído a cada uma. Depois, cada thread já recebe pouco trabalho, mas ainda precisa chegar às mesmas barreiras em todos os passos. A equipe só prossegue quando a última termina. Distribuição de blocos, sincronização e disputa por recursos passam a pesar mais em relação ao cálculo útil.

Isso explica por que aumentar a equipe pode produzir um patamar ou uma piora, em vez de reduzir o tempo continuamente. A distinção é importante: uma barreira faz parte da implementação e é um custo conhecido; afirmar que a RAM saturou exigiria outra evidência. Os vetores também podem ser atendidos por cache, e threads lógicas podem compartilhar recursos de um mesmo núcleo.

5.4 Cronômetro e entrada simplificados

O código chama `omp_get_wtime()` antes e depois do cálculo. A diferença fornece o tempo decorrido em segundos. As duas chamadas ficam na mesma thread, fora da região paralela. Todas as versões são compiladas com `-fopenmp`; na `v0`, a biblioteca fornece apenas o cronômetro, sem criar uma equipe.

Somente a `v0` recebe `[modo [campo.txt]]`, com 0 para a gaussiana, 1 para o campo parado e 2 para o constante. Esses modos permitem validar a referência sequencial. As versões `v1` a `v6` fixam a gaussiana na inicialização e recebem apenas `[campo.txt]` para exportação opcional. Assim, a comparação paralela mantém o mesmo campo sem repetir a seleção de modos em cada arquivo.

6 Tabela e gráfico para referência

A tabela e o gráfico preservam as medições anteriores à simplificação do cronômetro. A troca para `omp_get_wtime()` não foi acompanhada de uma nova medição de desempenho.

A Tabela 2 e a Figura 1 registram as medições com malha de 512×512 pontos e 500 passos, usando GCC 13.3.0 com -O2 em um Intel Core i7-14700HX sob WSL2. Os pontos até 25 threads usam medianas de cinco repetições após aquecimento; os de 26 a 28 usam dez repetições de confirmação.

Versão	1 thread	2 threads	4 threads	8 threads	14 threads	28 threads
v1 static	0,0970	0,0531	0,0289	0,0223	0,0207	0,0185
v2 static + collapse	0,1312	0,0722	0,0411	0,0326	0,0280	0,0488
v3 dynamic	0,1064	0,0621	0,0337	0,0220	0,0228	0,0276
v4 dynamic + collapse	0,2241	0,3662	0,3482	0,3342	0,3007	0,3225
v5 guided + collapse	0,1318	0,0706	0,0395	0,0277	0,0242	0,0329
v6 guided	0,1239	0,0588	0,0327	0,0240	0,0229	0,0269

Table 2: Tempo mediano em segundos, selecionado da varredura completa de 1 a 28 threads. A referência sequencial teve mediana de 0,1049 s.

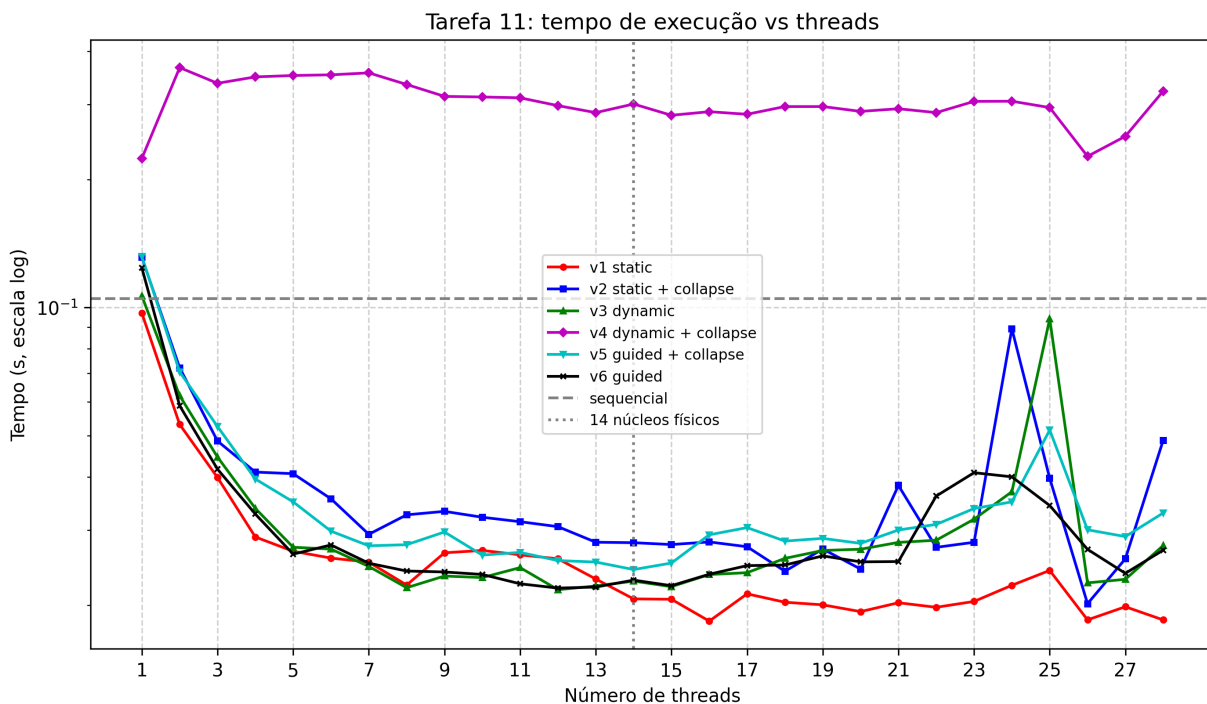


Figure 1: Tempo mediano das políticas com e sem `collapse(2)`, de 1 a 28 threads, em escala logarítmica.

Nos testes da v0, o campo parado continuou nulo e o constante permaneceu constante. As versões paralelas produziram o mesmo campo gaussiano da referência sequencial com 1, 2 e 4 threads. A comparação de desempenho usa a mesma malha e as mesmas opções de compilação. O cronômetro inclui o stencil e a sincronização; exclui inicialização, exportação e diagnósticos.

Associe cada tendência à comparação correspondente. A proximidade das versões por linhas se relaciona à carga uniforme. A curva alta de v4 é compatível com o custo de entregar muitos blocos pequenos. A diferença entre v4 e v5 relaciona-se ao uso de blocos fixos e decrescentes.

O patamar nas equipes maiores indica o peso das barreiras e dos recursos compartilhados, sem atribuir uma causa única ao tempo medido.

A Código-fonte

O apêndice reúne os sete códigos completos usados nos testes e nas medições.

```
1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v0_seq.c -lm -o ns_v0
2 #include <omp.h>
3 #include <math.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7
8 int main(int argc, char **argv) {
9     const int tamanho_grid = 512;
10    const int num_passos_tempo = 500;
11    if (argc > 3 || (argc > 1 &&
12        (argv[1][0] < '0' || argv[1][0] > '2' || argv[1][1] != '\0')) {
13        fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
14        return EXIT_FAILURE;
15    }
16    int mode = argc > 1 ? argv[1][0] - '0' : 0;
17    int threads = 1;
18    size_t count = (size_t)tamanho_grid * tamanho_grid;
19    double *u = malloc(count * sizeof(*u));
20    double *v = malloc(count * sizeof(*v));
21    if (!u || !v) {
22        fprintf(stderr, "Falha de alocao\n");
23        free(u); free(v);
24        return EXIT_FAILURE;
25    }
26    const double nu = 0.1, dt = 0.2, dx = 1.0;
27    const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
28    for (int i = 0; i < tamanho_grid; ++i) {
29        for (int j = 0; j < tamanho_grid; ++j) {
30            double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
31                - 1) / 2.0;
32            double sigma = tamanho_grid / 10.0;
33            int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
34                tamanho_grid-1;
35            u[(size_t)i*tamanho_grid+j] = mode == 2 ? 1.0 :
36                (mode == 1 || boundary ? 0.0 :
37                    0.1 * exp(-(x*x+y*y)/(2*sigma*sigma)));
38        }
39    }
40    /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
41    memcpy(v, u, count * sizeof(*v));
42    double start = omp_get_wtime();
43    for (int t = 0; t < num_passos_tempo; ++t) {
44        for (int i = 1; i < tamanho_grid-1; ++i) {
45            for (int j = 1; j < tamanho_grid-1; ++j) {
```

```

44         size_t c = (size_t)i*tamanho_grid+j;
45         v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c+
            tamanho_grid]-4*u[c]);
46     }
47 }
48     double *tmp = u; u = v; v = tmp;
49 }
50 double elapsed = omp_get_wtime() - start;
51 double peak = u[0], minimum = u[0], sum = 0.0;
52 for (size_t c = 0; c < count; ++c) {
53     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
        ];
54 }
55 printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
    =%.3f tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
56     "v0_seq", tamanho_grid, num_passos_tempo, mode, threads, r,
        elapsed, minimum, peak, sum);
57 int status = EXIT_SUCCESS;
58 if (argc == 3) {
59     FILE *file = fopen(argv[2], "w");
60     if (!file) { perror(argv[2]); status = EXIT_FAILURE; }
61     else {
62         for (size_t c = 0; c < count; ++c)
63             if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
                ' : '\n') < 0)
64                 status = EXIT_FAILURE;
65         if (fclose(file)) status = EXIT_FAILURE;
66     }
67 }
68 free(u); free(v);
69 return status;
70 }

```

Listing 9: v0_seq.c: referência sequencial

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp vi_static.c -lm -o ns_v1
2 #include <math.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <omp.h>
7
8 int main(int argc, char **argv) {
9     const int tamanho_grid = 512;
10    const int num_passos_tempo = 500;
11    if (argc > 2) {
12        fprintf(stderr, "Uso: %s [campo.txt]\n", argv[0]);
13        return EXIT_FAILURE;
14    }
15    int threads = 1;
16    size_t count = (size_t)tamanho_grid * tamanho_grid;
17    double *u = malloc(count * sizeof(*u));
18    double *v = malloc(count * sizeof(*v));
19    if (!u || !v) {
20        fprintf(stderr, "Falha de alocacao\n");
21        free(u); free(v);
22        return EXIT_FAILURE;
23    }
24    const double nu = 0.1, dt = 0.2, dx = 1.0;
25    const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
26    for (int i = 0; i < tamanho_grid; ++i) {
27        for (int j = 0; j < tamanho_grid; ++j) {
28            double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
29                - 1) / 2.0;
30            double sigma = tamanho_grid / 10.0;
31            int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
32                tamanho_grid-1;
33            u[(size_t)i*tamanho_grid+j] = boundary ? 0.0 :
34                0.1 * exp(-(x*x+y*y)/(2*sigma*sigma));
35        }
36    }
37    /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
38    memcpy(v, u, count * sizeof(*v));
39    double start = omp_get_wtime();
40    #pragma omp parallel default(none) shared(u,v,tamanho_grid,
41        num_passos_tempo,r,threads)
42    {
43        #pragma omp single
44        threads = omp_get_num_threads();
45        for (int t = 0; t < num_passos_tempo; ++t) {
46            #pragma omp for schedule(static)
47            for (int i = 1; i < tamanho_grid-1; ++i) {
48                for (int j = 1; j < tamanho_grid-1; ++j) {

```

```

46         size_t c = (size_t)i*tamanho_grid+j;
47         v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
           +tamanho_grid]-4*u[c]);
48     }
49     } /* Barreira: todas as escritas terminam antes da troca. */
50     #pragma omp single
51     {
52         double *tmp = u; u = v; v = tmp;
53     } /* Barreira: todas as threads veem a troca antes do próximo
           passo. */
54 }
55 }
56 double elapsed = omp_get_wtime() - start;
57 double peak = u[0], minimum = u[0], sum = 0.0;
58 for (size_t c = 0; c < count; ++c) {
59     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
        ];
60 }
61 printf("%s tamanho_grid=%d num_passos_tempo=%d threads=%d r=%.3f tempo
        =%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
62        "v1_static", tamanho_grid, num_passos_tempo, threads, r,
        elapsed, minimum, peak, sum);
63 int status = EXIT_SUCCESS;
64 if (argc == 2) {
65     FILE *file = fopen(argv[1], "w");
66     if (!file) { perror(argv[1]); status = EXIT_FAILURE; }
67     else {
68         for (size_t c = 0; c < count; ++c)
69             if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
                ' : '\n') < 0)
70                 status = EXIT_FAILURE;
71         if (fclose(file)) status = EXIT_FAILURE;
72     }
73 }
74 free(u); free(v);
75 return status;
76 }

```

Listing 10: v1_static.c: static por linhas

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v2_static_collapse.c -lm -o
  ns_v2
2 #include <math.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <omp.h>
7
8 int main(int argc, char **argv) {
9     const int tamanho_grid = 512;
10    const int num_passos_tempo = 500;
11    if (argc > 2) {
12        fprintf(stderr, "Uso: %s [campo.txt]\n", argv[0]);
13        return EXIT_FAILURE;
14    }
15    int threads = 1;
16    size_t count = (size_t)tamanho_grid * tamanho_grid;
17    double *u = malloc(count * sizeof(*u));
18    double *v = malloc(count * sizeof(*v));
19    if (!u || !v) {
20        fprintf(stderr, "Falha de alocacao\n");
21        free(u); free(v);
22        return EXIT_FAILURE;
23    }
24    const double nu = 0.1, dt = 0.2, dx = 1.0;
25    const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
26    for (int i = 0; i < tamanho_grid; ++i) {
27        for (int j = 0; j < tamanho_grid; ++j) {
28            double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
                - 1) / 2.0;
29            double sigma = tamanho_grid / 10.0;
30            int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
                tamanho_grid-1;
31            u[(size_t)i*tamanho_grid+j] = boundary ? 0.0 :
32                0.1 * exp(-(x*x+y*y)/(2*sigma*sigma));
33        }
34    }
35    /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
36    memcpy(v, u, count * sizeof(*v));
37    double start = omp_get_wtime();
38    #pragma omp parallel default(none) shared(u,v,tamanho_grid,
        num_passos_tempo,r,threads)
39    {
40        #pragma omp single
41        threads = omp_get_num_threads();
42        for (int t = 0; t < num_passos_tempo; ++t) {
43            #pragma omp for collapse(2) schedule(static)
44            for (int i = 1; i < tamanho_grid-1; ++i) {

```

```

45         for (int j = 1; j < tamanho_grid-1; ++j) {
46             size_t c = (size_t)i*tamanho_grid+j;
47             v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
               +tamanho_grid]-4*u[c]);
48         }
49     } /* Barreira: todas as escritas terminam antes da troca. */
50     #pragma omp single
51     {
52         double *tmp = u; u = v; v = tmp;
53     } /* Barreira: todas as threads veem a troca antes do próximo
           passo. */
54 }
55 }
56 double elapsed = omp_get_wtime() - start;
57 double peak = u[0], minimum = u[0], sum = 0.0;
58 for (size_t c = 0; c < count; ++c) {
59     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
        ];
60 }
61 printf("%s tamanho_grid=%d num_passos_tempo=%d threads=%d r=%.3f tempo
        =%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
62        "v2_static_collapse", tamanho_grid, num_passos_tempo, threads,
        r, elapsed, minimum, peak, sum);
63 int status = EXIT_SUCCESS;
64 if (argc == 2) {
65     FILE *file = fopen(argv[1], "w");
66     if (!file) { perror(argv[1]); status = EXIT_FAILURE; }
67     else {
68         for (size_t c = 0; c < count; ++c)
69             if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
                ', : '\n') < 0)
70                 status = EXIT_FAILURE;
71         if (fclose(file)) status = EXIT_FAILURE;
72     }
73 }
74 free(u); free(v);
75 return status;
76 }

```

Listing 11: v2_static_collapse.c: static por células

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v3_dynamic.c -lm -o ns_v3
2 #include <math.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <omp.h>
7
8 int main(int argc, char **argv) {
9     const int tamanho_grid = 512;
10    const int num_passos_tempo = 500;
11    if (argc > 2) {
12        fprintf(stderr, "Uso: %s [campo.txt]\n", argv[0]);
13        return EXIT_FAILURE;
14    }
15    int threads = 1;
16    size_t count = (size_t)tamanho_grid * tamanho_grid;
17    double *u = malloc(count * sizeof(*u));
18    double *v = malloc(count * sizeof(*v));
19    if (!u || !v) {
20        fprintf(stderr, "Falha de alocação\n");
21        free(u); free(v);
22        return EXIT_FAILURE;
23    }
24    const double nu = 0.1, dt = 0.2, dx = 1.0;
25    const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
26    for (int i = 0; i < tamanho_grid; ++i) {
27        for (int j = 0; j < tamanho_grid; ++j) {
28            double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
29                - 1) / 2.0;
30            double sigma = tamanho_grid / 10.0;
31            int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
32                tamanho_grid-1;
33            u[(size_t)i*tamanho_grid+j] = boundary ? 0.0 :
34                0.1 * exp(-(x*x+y*y)/(2*sigma*sigma));
35        }
36    }
37    /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
38    memcpy(v, u, count * sizeof(*v));
39    double start = omp_get_wtime();
40    #pragma omp parallel default(none) shared(u,v,tamanho_grid,
41        num_passos_tempo,r,threads)
42    {
43        #pragma omp single
44        threads = omp_get_num_threads();
45        for (int t = 0; t < num_passos_tempo; ++t) {
46            #pragma omp for schedule(dynamic, 16)
47            for (int i = 1; i < tamanho_grid-1; ++i) {
48                for (int j = 1; j < tamanho_grid-1; ++j) {

```

```

46         size_t c = (size_t)i*tamanho_grid+j;
47         v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
          +tamanho_grid]-4*u[c]);
48     }
49     } /* Barreira: todas as escritas terminam antes da troca. */
50     #pragma omp single
51     {
52         double *tmp = u; u = v; v = tmp;
53     } /* Barreira: todas as threads veem a troca antes do próximo
          passo. */
54 }
55 }
56 double elapsed = omp_get_wtime() - start;
57 double peak = u[0], minimum = u[0], sum = 0.0;
58 for (size_t c = 0; c < count; ++c) {
59     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
        ];
60 }
61 printf("%s tamanho_grid=%d num_passos_tempo=%d threads=%d r=%.3f tempo
        =%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
62        "v3_dynamic", tamanho_grid, num_passos_tempo, threads, r,
        elapsed, minimum, peak, sum);
63 int status = EXIT_SUCCESS;
64 if (argc == 2) {
65     FILE *file = fopen(argv[1], "w");
66     if (!file) { perror(argv[1]); status = EXIT_FAILURE; }
67     else {
68         for (size_t c = 0; c < count; ++c)
69             if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
                ' : '\n') < 0)
70                 status = EXIT_FAILURE;
71         if (fclose(file)) status = EXIT_FAILURE;
72     }
73 }
74 free(u); free(v);
75 return status;
76 }

```

Listing 12: v3_dynamic.c: dynamic por linhas

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v4_dynamic_collapse.c -lm -o
   ns_v4
2 #include <math.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <omp.h>
7
8 int main(int argc, char **argv) {
9     const int tamanho_grid = 512;
10    const int num_passos_tempo = 500;
11    if (argc > 2) {
12        fprintf(stderr, "Uso: %s [campo.txt]\n", argv[0]);
13        return EXIT_FAILURE;
14    }
15    int threads = 1;
16    size_t count = (size_t)tamanho_grid * tamanho_grid;
17    double *u = malloc(count * sizeof(*u));
18    double *v = malloc(count * sizeof(*v));
19    if (!u || !v) {
20        fprintf(stderr, "Falha de alocacao\n");
21        free(u); free(v);
22        return EXIT_FAILURE;
23    }
24    const double nu = 0.1, dt = 0.2, dx = 1.0;
25    const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
26    for (int i = 0; i < tamanho_grid; ++i) {
27        for (int j = 0; j < tamanho_grid; ++j) {
28            double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
                - 1) / 2.0;
29            double sigma = tamanho_grid / 10.0;
30            int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
                tamanho_grid-1;
31            u[(size_t)i*tamanho_grid+j] = boundary ? 0.0 :
32                0.1 * exp(-(x*x+y*y)/(2*sigma*sigma));
33        }
34    }
35    /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
36    memcpy(v, u, count * sizeof(*v));
37    double start = omp_get_wtime();
38    #pragma omp parallel default(none) shared(u,v,tamanho_grid,
        num_passos_tempo,r,threads)
39    {
40        #pragma omp single
41        threads = omp_get_num_threads();
42        for (int t = 0; t < num_passos_tempo; ++t) {
43            #pragma omp for collapse(2) schedule(dynamic, 16)
44            for (int i = 1; i < tamanho_grid-1; ++i) {

```

```

45         for (int j = 1; j < tamanho_grid-1; ++j) {
46             size_t c = (size_t)i*tamanho_grid+j;
47             v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
               +tamanho_grid]-4*u[c]);
48         }
49     } /* Barreira: todas as escritas terminam antes da troca. */
50     #pragma omp single
51     {
52         double *tmp = u; u = v; v = tmp;
53     } /* Barreira: todas as threads veem a troca antes do próximo
           passo. */
54 }
55 }
56 double elapsed = omp_get_wtime() - start;
57 double peak = u[0], minimum = u[0], sum = 0.0;
58 for (size_t c = 0; c < count; ++c) {
59     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
        ];
60 }
61 printf("%s tamanho_grid=%d num_passos_tempo=%d threads=%d r=%.3f tempo
        =%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
62        "v4_dynamic_collapse", tamanho_grid, num_passos_tempo, threads,
        r, elapsed, minimum, peak, sum);
63 int status = EXIT_SUCCESS;
64 if (argc == 2) {
65     FILE *file = fopen(argv[1], "w");
66     if (!file) { perror(argv[1]); status = EXIT_FAILURE; }
67     else {
68         for (size_t c = 0; c < count; ++c)
69             if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
                ', : '\n') < 0)
70                 status = EXIT_FAILURE;
71         if (fclose(file)) status = EXIT_FAILURE;
72     }
73 }
74 free(u); free(v);
75 return status;
76 }

```

Listing 13: v4_dynamic_collapse.c: dynamic por células

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v5_guided_collapse.c -lm -o
   ns_v5
2 #include <math.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <omp.h>
7
8 int main(int argc, char **argv) {
9     const int tamanho_grid = 512;
10    const int num_passos_tempo = 500;
11    if (argc > 2) {
12        fprintf(stderr, "Uso: %s [campo.txt]\n", argv[0]);
13        return EXIT_FAILURE;
14    }
15    int threads = 1;
16    size_t count = (size_t)tamanho_grid * tamanho_grid;
17    double *u = malloc(count * sizeof(*u));
18    double *v = malloc(count * sizeof(*v));
19    if (!u || !v) {
20        fprintf(stderr, "Falha de alocacao\n");
21        free(u); free(v);
22        return EXIT_FAILURE;
23    }
24    const double nu = 0.1, dt = 0.2, dx = 1.0;
25    const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
26    for (int i = 0; i < tamanho_grid; ++i) {
27        for (int j = 0; j < tamanho_grid; ++j) {
28            double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
                - 1) / 2.0;
29            double sigma = tamanho_grid / 10.0;
30            int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
                tamanho_grid-1;
31            u[(size_t)i*tamanho_grid+j] = boundary ? 0.0 :
32                0.1 * exp(-(x*x+y*y)/(2*sigma*sigma));
33        }
34    }
35    /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
36    memcpy(v, u, count * sizeof(*v));
37    double start = omp_get_wtime();
38    #pragma omp parallel default(none) shared(u,v,tamanho_grid,
        num_passos_tempo,r,threads)
39    {
40        #pragma omp single
41        threads = omp_get_num_threads();
42        for (int t = 0; t < num_passos_tempo; ++t) {
43            #pragma omp for collapse(2) schedule(guided, 16)
44            for (int i = 1; i < tamanho_grid-1; ++i) {

```

```

45         for (int j = 1; j < tamanho_grid-1; ++j) {
46             size_t c = (size_t)i*tamanho_grid+j;
47             v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
               +tamanho_grid]-4*u[c]);
48         }
49     } /* Barreira: todas as escritas terminam antes da troca. */
50     #pragma omp single
51     {
52         double *tmp = u; u = v; v = tmp;
53     } /* Barreira: todas as threads veem a troca antes do próximo
           passo. */
54 }
55 }
56 double elapsed = omp_get_wtime() - start;
57 double peak = u[0], minimum = u[0], sum = 0.0;
58 for (size_t c = 0; c < count; ++c) {
59     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
        ];
60 }
61 printf("%s tamanho_grid=%d num_passos_tempo=%d threads=%d r=%.3f tempo
        =%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
62        "v5_guided_collapse", tamanho_grid, num_passos_tempo, threads,
        r, elapsed, minimum, peak, sum);
63 int status = EXIT_SUCCESS;
64 if (argc == 2) {
65     FILE *file = fopen(argv[1], "w");
66     if (!file) { perror(argv[1]); status = EXIT_FAILURE; }
67     else {
68         for (size_t c = 0; c < count; ++c)
69             if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
                ', : '\n') < 0)
70                 status = EXIT_FAILURE;
71         if (fclose(file)) status = EXIT_FAILURE;
72     }
73 }
74 free(u); free(v);
75 return status;
76 }

```

Listing 14: v5_guided_collapse.c: guided por células

```

1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v6_guided.c -lm -o ns_v6
2 #include <math.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <omp.h>
7
8 int main(int argc, char **argv) {
9     const int tamanho_grid = 512;
10    const int num_passos_tempo = 500;
11    if (argc > 2) {
12        fprintf(stderr, "Uso: %s [campo.txt]\n", argv[0]);
13        return EXIT_FAILURE;
14    }
15    int threads = 1;
16    size_t count = (size_t)tamanho_grid * tamanho_grid;
17    double *u = malloc(count * sizeof(*u));
18    double *v = malloc(count * sizeof(*v));
19    if (!u || !v) {
20        fprintf(stderr, "Falha de alocao\n");
21        free(u); free(v);
22        return EXIT_FAILURE;
23    }
24    const double nu = 0.1, dt = 0.2, dx = 1.0;
25    const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
26    for (int i = 0; i < tamanho_grid; ++i) {
27        for (int j = 0; j < tamanho_grid; ++j) {
28            double x = i - (tamanho_grid - 1) / 2.0, y = j - (tamanho_grid
29                - 1) / 2.0;
30            double sigma = tamanho_grid / 10.0;
31            int boundary = i == 0 || j == 0 || i == tamanho_grid-1 || j ==
32                tamanho_grid-1;
33            u[(size_t)i*tamanho_grid+j] = boundary ? 0.0 :
34                0.1 * exp(-(x*x+y*y)/(2*sigma*sigma));
35        }
36    }
37    /* Bordas fixas iguais nos dois buffers: nenhuma escrita posterior. */
38    memcpy(v, u, count * sizeof(*v));
39    double start = omp_get_wtime();
40    #pragma omp parallel default(none) shared(u,v,tamanho_grid,
41        num_passos_tempo,r,threads)
42    {
43        #pragma omp single
44        threads = omp_get_num_threads();
45        for (int t = 0; t < num_passos_tempo; ++t) {
46            #pragma omp for schedule(guided, 16)
47            for (int i = 1; i < tamanho_grid-1; ++i) {
48                for (int j = 1; j < tamanho_grid-1; ++j) {

```

```

46         size_t c = (size_t)i*tamanho_grid+j;
47         v[c] = u[c] + r * (u[c-1]+u[c+1]+u[c-tamanho_grid]+u[c
           +tamanho_grid]-4*u[c]);
48     }
49     } /* Barreira: todas as escritas terminam antes da troca. */
50     #pragma omp single
51     {
52         double *tmp = u; u = v; v = tmp;
53     } /* Barreira: todas as threads veem a troca antes do próximo
           passo. */
54 }
55 }
56 double elapsed = omp_get_wtime() - start;
57 double peak = u[0], minimum = u[0], sum = 0.0;
58 for (size_t c = 0; c < count; ++c) {
59     peak = fmax(peak, u[c]); minimum = fmin(minimum, u[c]); sum += u[c
        ];
60 }
61 printf("%s tamanho_grid=%d num_passos_tempo=%d threads=%d r=%.3f tempo
        =%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
62        "v6_guided", tamanho_grid, num_passos_tempo, threads, r,
        elapsed, minimum, peak, sum);
63 int status = EXIT_SUCCESS;
64 if (argc == 2) {
65     FILE *file = fopen(argv[1], "w");
66     if (!file) { perror(argv[1]); status = EXIT_FAILURE; }
67     else {
68         for (size_t c = 0; c < count; ++c)
69             if (fprintf(file, "%.17g%c", u[c], (c+1)%tamanho_grid ? '
                ' : '\n') < 0)
70                 status = EXIT_FAILURE;
71         if (fclose(file)) status = EXIT_FAILURE;
72     }
73 }
74 free(u); free(v);
75 return status;
76 }

```

Listing 15: v6_guided.c: guided por linhas